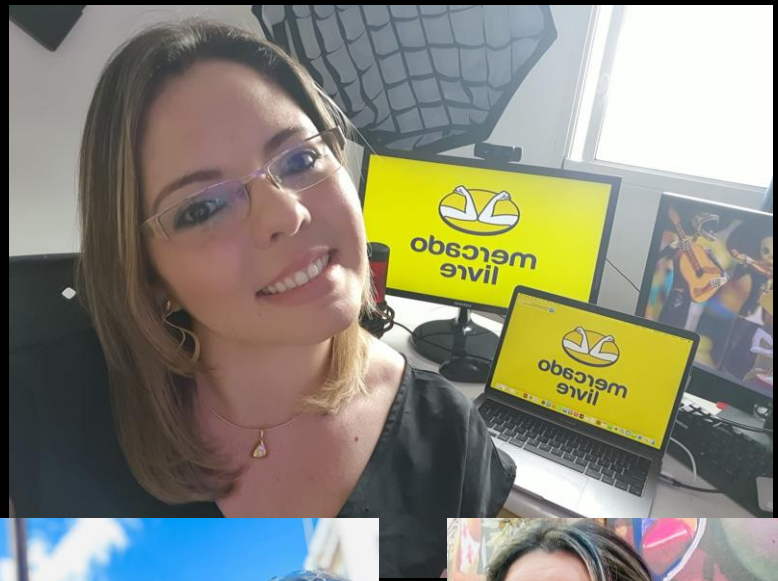

FIAP
MBA

Deep Learning & Redes Neurais Modernas

Aula 01 | Redes profundas e Visão Computacional. Como uma máquina aprende a partir do erro e como ela passou a enxergar.

QUEM SOU EU

- **Bacharel** em Estatística pela UFSCAR
- **Mestre e Doutora** em Estatística pelo ICMC-USP
- **Pesquisadora** na área de Modelos Preditivos e Análise de Sobrevida
- **Consultora** na área de Inteligência de Negócios
- **Professora** de pós-graduação na área de Machine Learning e Mineração de Dados
- **Fundei uma empresa de IA** - FLAI Inteligência Artificial.
- **Cientista de dados**: Itaú , Mercado Livre, SKY Brasil







Roteiro da aula

4 horas. Você executa os notebooks junto comigo, no Google Colab.

65 min	20 min	50 min	15 min	45 min	15 min	25 min
Intuição	**Demo de conceito**	**Aplicação 1**	**Intervalo**	**Aplicação 2**	**Momento uau**	**Case e discussão**
Como uma rede aprende; como ela enxerga.	Gradiente, ativação e backprop em código.	Uma CNN classificando 10 tipos de objeto.		Detectar e contar objetos em fotos e vídeo.	Transfer learning: 100 fotos por classe.	Onde gera valor, o que custa, quando não usar.

Mais 5 min de gancho para a aula 2. **Notebooks:** 01_demo_conceito, 02_aplicacao1_cnn_cifar10, 03_aplicacao2_deteccao_objetos, 04_uau_transfer_learning_cifar10.

A PERGUNTA DESTA AULA

Como uma máquina aprende a partir do erro?

E por que, em uma década, ensinar computadores a enxergar deixou de ser pesquisa e virou produto de prateleira.

AO FINAL DA AULA VOCÊ VAI SABER

- Explicar backpropagation para um diretor sem escrever uma equação
- Ler uma curva de treino e uma matriz de confusão e dizer o que fazer com elas
- Decidir se um problema da sua empresa é de classificação, detecção ou nenhum dos dois

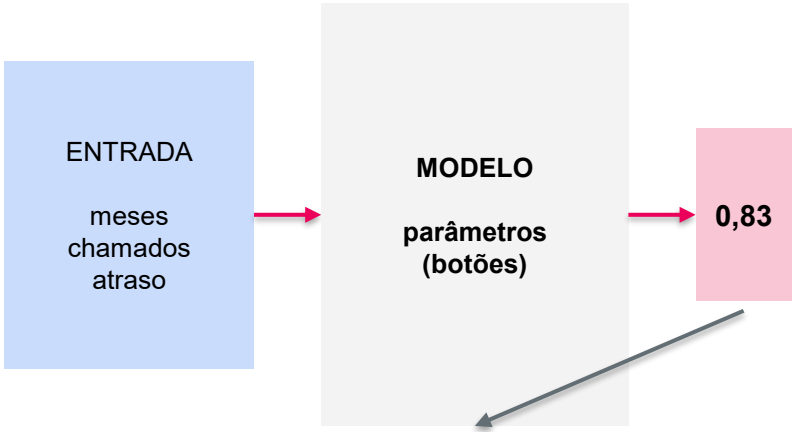
PARTE 1

Como uma rede aprende

Neurônio, camadas, perda, gradiente e backpropagation. Sem notação pesada: a mecânica por trás de qualquer rede, da mais simples ao GPT.

Um modelo é uma função com botões

- **Entrada:** os dados que descrevem o caso (meses de contrato, chamados ao SAC, atraso no pagamento).
- **Parâmetros:** os "botões" internos. Uma rede pequena tem milhares; um LLM, bilhões.
- **Saída:** a previsão (probabilidade de cancelar).
- **Aprender:** girar os botões até a saída errar menos nos exemplos que já conhecemos.

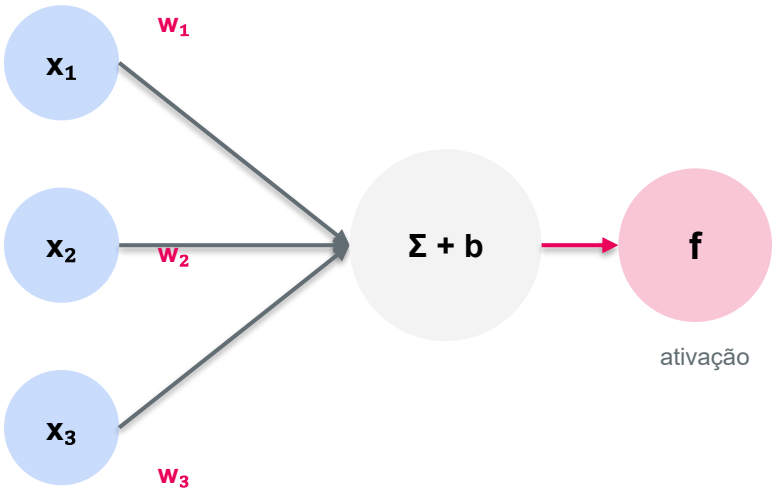


O erro (0,83 contra "não cancelou") volta para ajustar os botões.

Ninguém programa a regra. A regra emerge do ajuste dos botões aos dados.

O neurônio: um comitê que vota com pesos

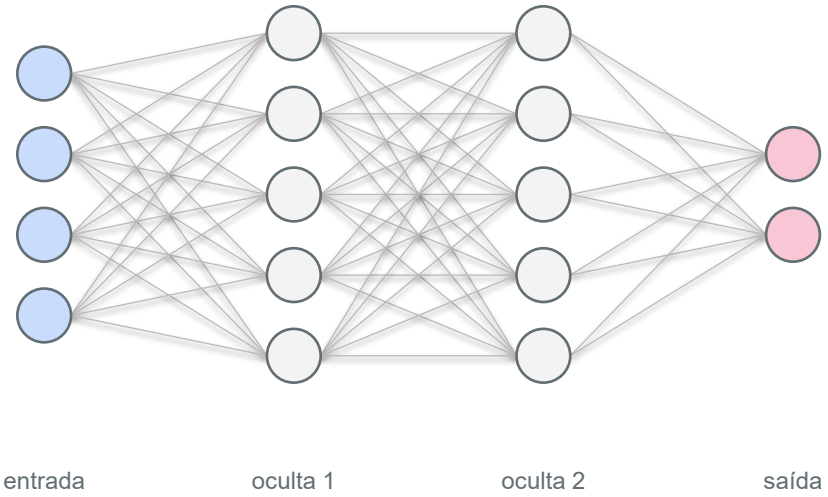
- **Soma ponderada:** cada entrada entra multiplicada por um peso, a sua importância.
- **Viés:** a tendência de base, antes de olhar qualquer entrada.
- **Ativação:** a "dobra" aplicada ao resultado. Sem ela, o neurônio só desenha uma reta.
- **Sozinho,** um neurônio é uma regressão logística. A força vem de juntar muitos.



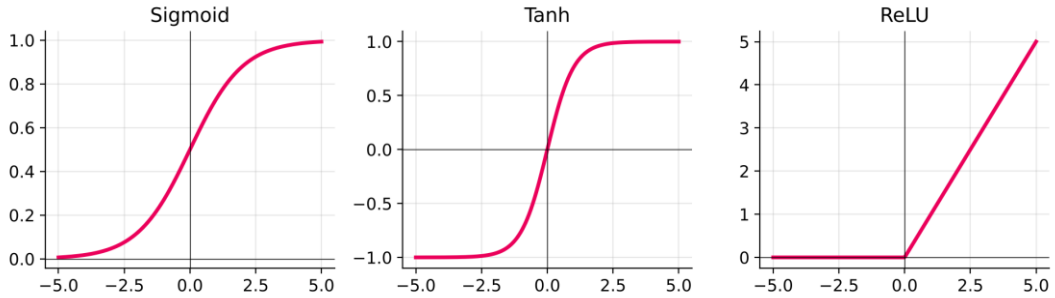
$$y = f(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

Camadas: empilhar neurônios cria abstrações

- **Camada de entrada:** recebe os dados brutos, um neurônio por variável.
- **Camadas ocultas:** cada uma transforma a saída da anterior em algo mais abstrato.
- **Camada de saída:** decide: um número (regressão) ou uma probabilidade por classe.
- **"Profundo"** é só isso: muitas camadas ocultas. Deep learning = redes com várias camadas.
- **Parâmetros = conexões.** Ao lado: 4·5 + 5·5 + 5·2 = 55 pesos, mais 12 vieses.



Sem a "dobra", dez camadas valem uma



- Empilhar operações lineares dá uma operação linear: uma matriz só faz o mesmo trabalho.
- A função de ativação introduz a não linearidade que permite fronteiras curvas.
- **ReLU,** max(0, z): a padrão hoje. Barata de calcular e funciona.

NA DEMO VOCÊ VAI PROVAR ISSO

Três matrizes aplicadas em sequência a um vetor dão exatamente o mesmo resultado que uma única matriz.

```
np.allclose(W3 @ W2 @ W1 @ x, W @ x)
→  True
```

Profundidade sem não linearidade é desperdício de computação.

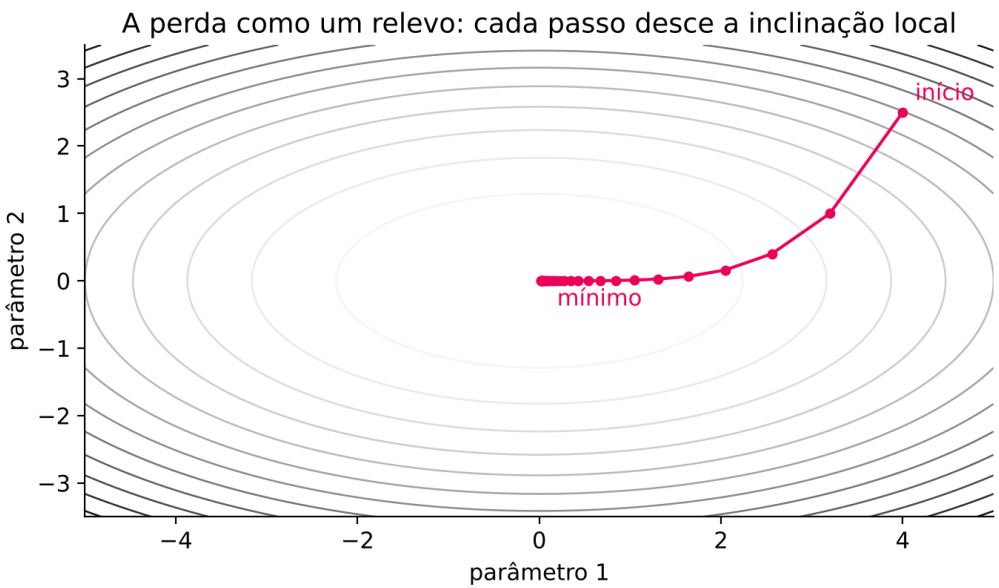
A função de perda: o número que diz quanto erramos

Treinar é minimizar a perda nos exemplos de treino. A perda é a montanha; os parâmetros são a posição.

Tarefa	Perda típica	Lê-se como
Regressão: prever um valor (demanda, preço)	**Erro quadrático médio**	Distância média entre previsão e valor real, elevada ao quadrado: erros grandes pesam muito
Classificação: prever uma classe (fraude, defeito, gato)	**Entropia cruzada**	Quanto o modelo estava confiante no palpite errado. Errar com 99% de certeza custa muito mais que errar com 55%

A escolha da perda é uma decisão de negócio disfarçada de matemática: ela define o que é "errar feio".

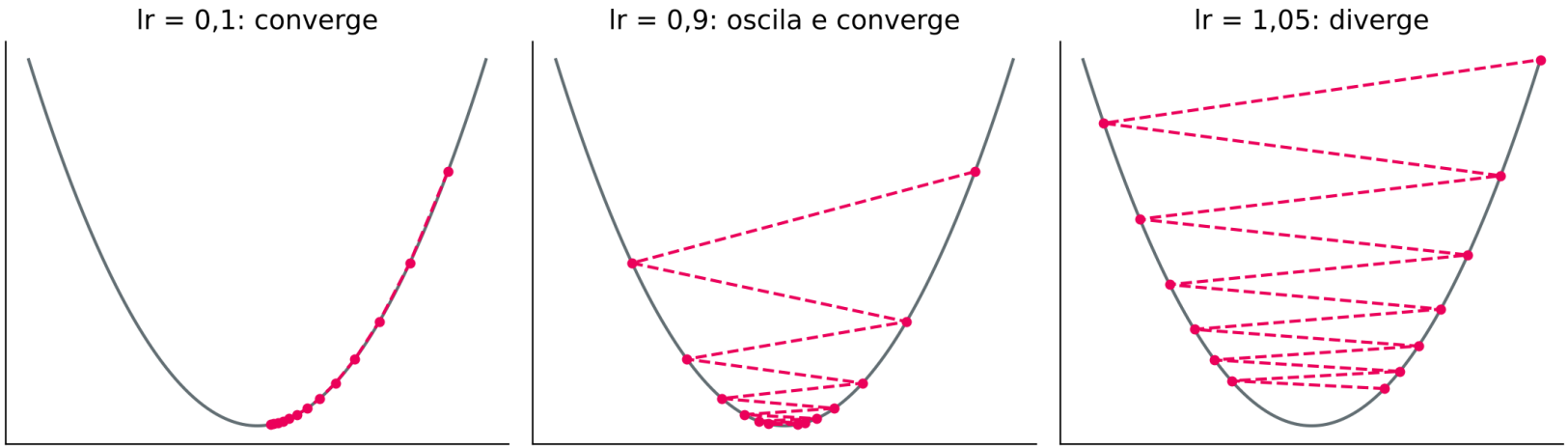
Descida do gradiente: descer a montanha no nevoeiro



- **1. Sentir a inclinação.** O gradiente diz em que direção o erro cresce mais rápido.
- **2. Dar um passo contra ela.** parâmetro = parâmetro - learning rate × gradiente.
- **3. Repetir** milhares de vezes, até o erro parar de cair.
- **Perto do vale a inclinação some** e os passos encolhem sozinhos.

Não há mapa. O algoritmo nunca vê a montanha inteira, só a inclinação onde está.

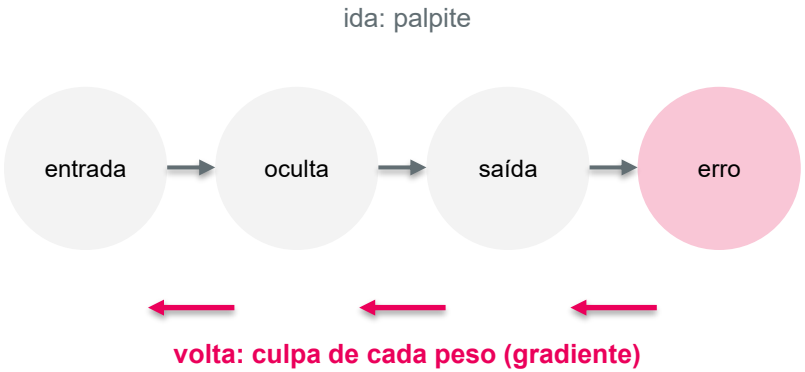
Learning rate: o tamanho do passo decide o destino



- **"A perda virou NaN"** = passo grande demais: o modelo saltou para fora da montanha.
- **"Treina há dias e não melhora"** = passo pequeno demais, ou preso num vale raso.
- **É o hiperparâmetro mais importante de qualquer rede.** Na prática usa-se um passo que diminui com o tempo.

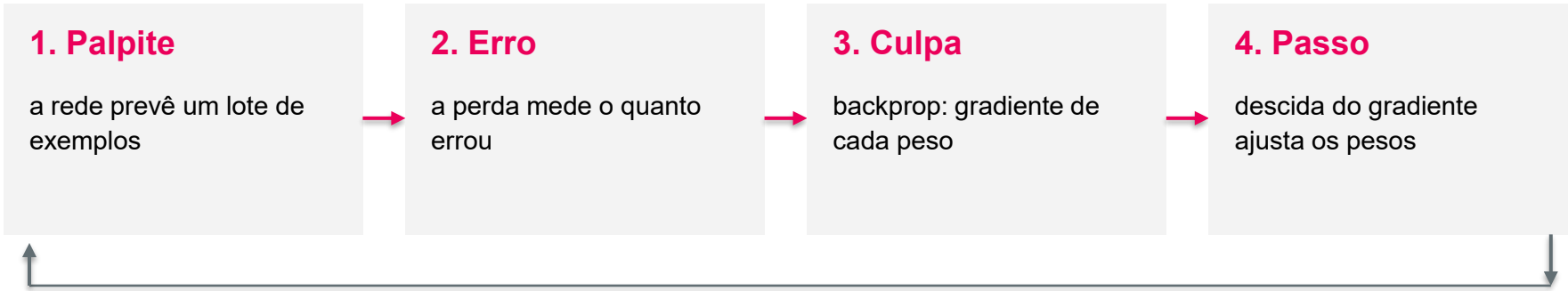
Backpropagation: de quem é a culpa pelo erro?

- Para ajustar um peso preciso saber quanto ele contribuiu para o erro.
- **O backprop calcula isso da saída para a entrada,** camada a camada, distribuindo a "culpa" pela regra da cadeia.
- **O resultado é um gradiente por peso:** milhões de números, calculados numa passada.
- **Publicado em 1986** (Rumelhart, Hinton e Williams). Só virou prática com dados e GPUs, 25 anos depois.



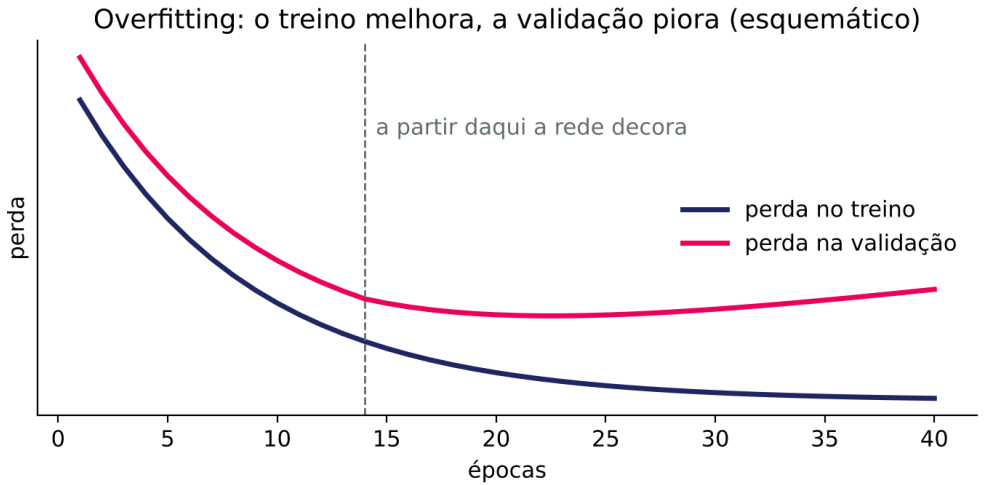
Na demo: loss.backward() e o gradiente de cada peso impresso na tela.

O ciclo de treino: palpite, erro, culpa, passo



- **Lote (batch):** quantos exemplos entram por passo. 128 na nossa aplicação.
- **Época:** uma passada por todos os exemplos de treino. Treinos reais levam dezenas a centenas.
- **Milhões de repetições.** É por isso que a GPU importa: ela faz as contas de muitos exemplos ao mesmo tempo.

Treino, validação e teste: a única forma honesta de medir



- **Treino:** ajusta os pesos.
- **Validação:** escolhe a melhor época e os hiperparâmetros. A rede não aprende com ela.
- **Teste:** intocado até o fim. É a estimativa do que vai acontecer em produção.
- **Overfitting:** a rede decora o treino. Treino melhora, validação piora.
- **Remédios:** mais dados, dropout, parar cedo, modelo menor.

Um modelo com 99% no treino e 70% na validação não "quase funciona". Ele decorou.

O que decidir antes de treinar qualquer rede

Dados rotulados

Quantos exemplos existem? Quem rotula, a que custo? Rotular costuma ser a etapa mais cara e lenta.

Métrica de negócio

O que é um erro caro? Um falso positivo e um falso negativo raramente custam o mesmo.

Computação

CPU basta para o piloto? Quanto tempo por treino? GPU muda o jogo, mas tem custo.

Baseline

Um modelo simples primeiro. Se a regra de negócio ou a regressão logística resolve, a rede não se paga.

Estas quatro perguntas valem para a CNN de hoje e para o LLM da aula 4.

DEMO DE CONCEITO | 20 MIN

Ver a rede aprender, uma célula por vez

Abra o notebook 01_demo_conceito_gradiente_backprop. Execute cada célula junto comigo (Shift+Enter).

- Descida do gradiente em f(x) = x² e o efeito do learning rate (converge, oscila, diverge)
- Três camadas lineares valem uma: a prova em três linhas
- Backprop visível: o gradiente de cada peso impresso, e a perda caindo após um passo
- A fronteira de decisão sendo esculpida época a época

PARTE 2

Ensinar máquinas a enxergar

Da matriz de pixels à rede convolucional. E da pergunta "o que há na foto?" para "o que há, e onde?".

Para a rede, uma foto é uma caixa de números

foto 32 x 32 pixels (classe: carro)



recorte 8 x 8 do canal vermelho (0 a 255)

211	215	176	124	119	148	157	173
151	150	160	144	129	139	122	133
153	126	165	186	156	133	162	161
253	227	238	244	214	188	188	186
255	255	255	255	242	244	213	206
194	218	245	255	255	255	255	248
201	212	209	221	254	255	255	255
198	202	199	202	198	172	171	227

- **32 × 32 pixels × 3 canais** (vermelho, verde, azul) = 3.072 números entre 0 e 255.
- **Uma foto Full HD:** 1920 × 1080 × 3 = 6,2 milhões de números.
- **Nenhum número diz "carro".** O padrão entre eles diz.
- **Normalizar** (0 a 255 → valores pequenos em torno de zero) é o primeiro passo de qualquer pipeline.

Por que uma rede comum falha com imagens

3.072

entradas numa foto 32 × 32

393 mil

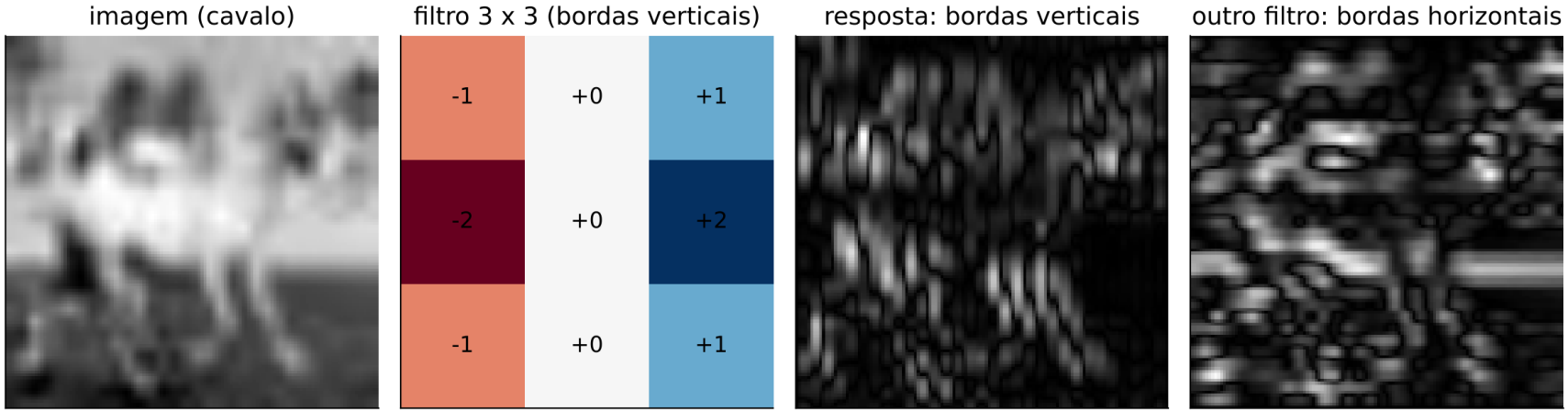
pesos só na primeira camada, com 128 neurônios

796 milhões

pesos na primeira camada se a foto for Full HD

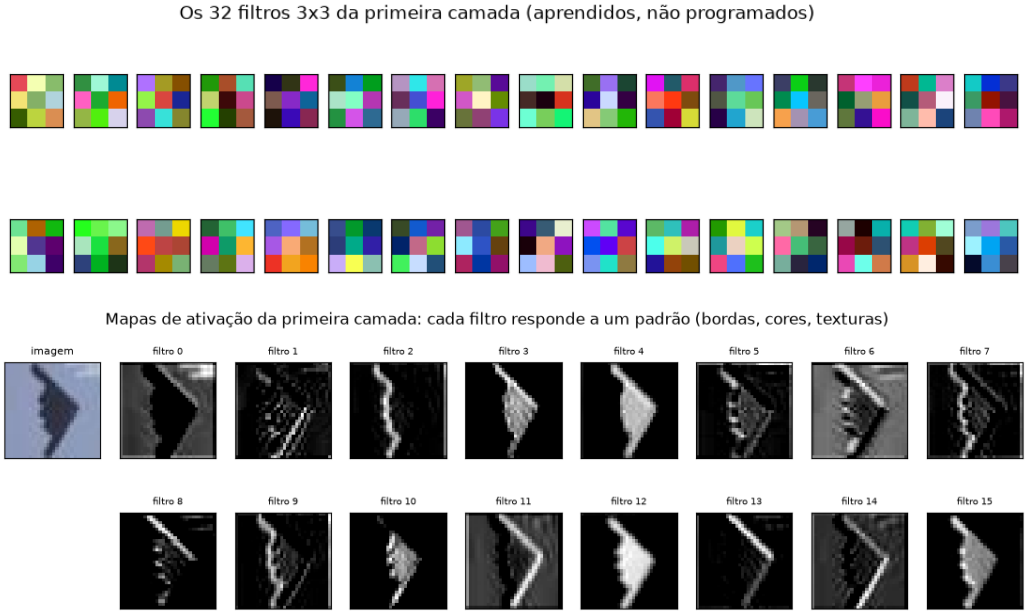
- **Cada pixel conectado a cada neurônio:** o número de pesos explode e exige milhões de fotos rotuladas.
- **A posição vira problema:** o mesmo gato deslocado 10 pixels à direita é, para a rede densa, uma entrada completamente diferente.
- **O que precisamos:** procurar o mesmo padrão em qualquer posição da imagem, com poucos pesos. É isso que a convolução faz.

Convolução: um filtro pequeno varre a imagem



- **Um filtro 3 × 3 tem 9 pesos.** Ele desliza por toda a imagem e responde forte onde encontra o seu padrão.
- **O mesmo filtro em todo lugar:** o padrão é reconhecido em qualquer posição, com 9 pesos em vez de milhares.
- **Ninguém desenha os filtros.** O filtro de borda acima foi feito à mão para explicar; os da rede são aprendidos pelo gradiente.

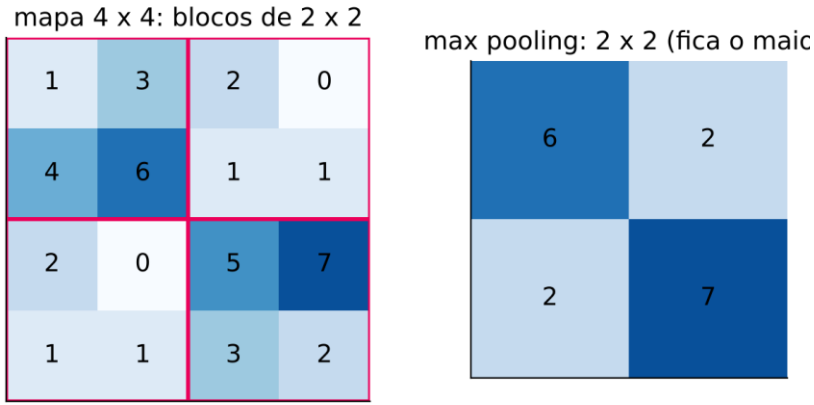
Hierarquia: bordas viram texturas, texturas viram partes



Figuras geradas pela execução do notebook 02 (10 épocas, CPU).

- **Camada 1:** bordas, cores, contrastes.
- **Camada 2:** combina bordas em texturas e cantos.
- **Camada 3:** combina texturas em partes de objetos (roda, orelha, asa).
- **Camadas densas:** juntam as partes e decidem a classe.
- **Ao lado,** os 32 filtros reais da rede que vamos treinar hoje e as respostas deles a uma imagem de teste.

Pooling: encolher sem perder o essencial



- **Max pooling 2 × 2:** cada bloco de quatro valores vira o maior deles.
- **A imagem encolhe pela metade** a cada etapa: 32 → 16 → 8 → 4. Menos contas nas camadas seguintes.
- **Tolerância a deslocamentos:** se a borda se move um pixel, o máximo do bloco não muda.
- **O que importa fica;** o detalhe fino vai embora. É uma troca deliberada.

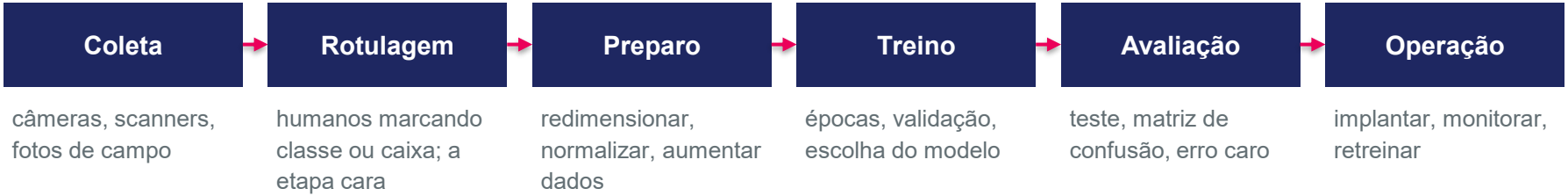
A arquitetura da aplicação 1, camada por camada

Camada	O que faz	Saída	Parâmetros
conv1 (32 filtros 3×3)	bordas e cores	32 × 32 × 32	896
max pool 2×2	encolhe	32 × 16 × 16	0
conv2 (64 filtros)	texturas	64 × 16 × 16	18.496
max pool 2×2	encolhe	64 × 8 × 8	0
conv3 (128 filtros)	partes de objetos	128 × 8 × 8	73.856
max pool 2×2	encolhe	128 × 4 × 4	0
flatten + dropout	vira um vetor de 2.048	2.048	0
densa (128) + ReLU	combina as partes	128	262.272
densa (10)	decide a classe	10	1.290

TOTAL: 356.810 parâmetros. A camada densa tem mais pesos que todas as convoluções juntas.

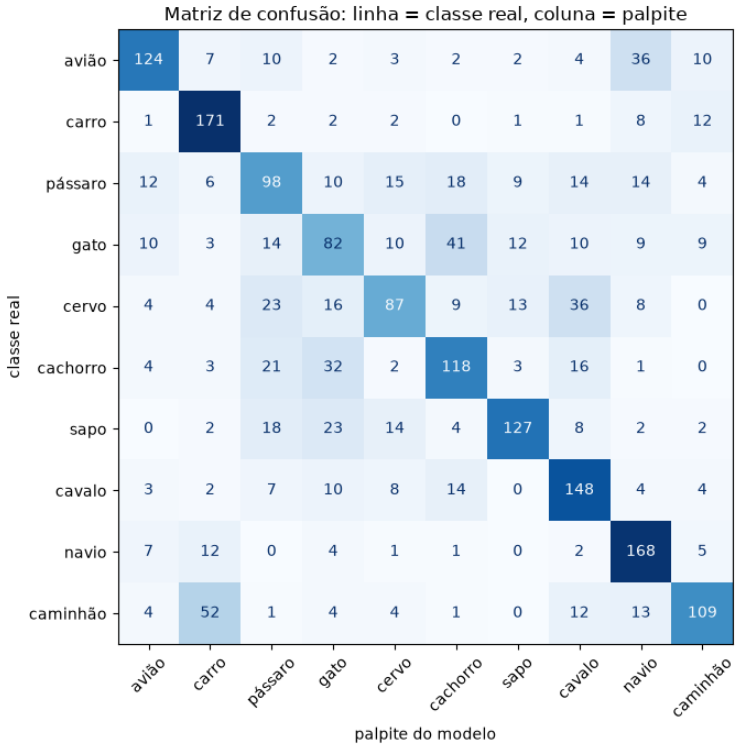
Convoluções são baratas porque reutilizam o mesmo filtro na imagem inteira.

O pipeline de um projeto de visão computacional



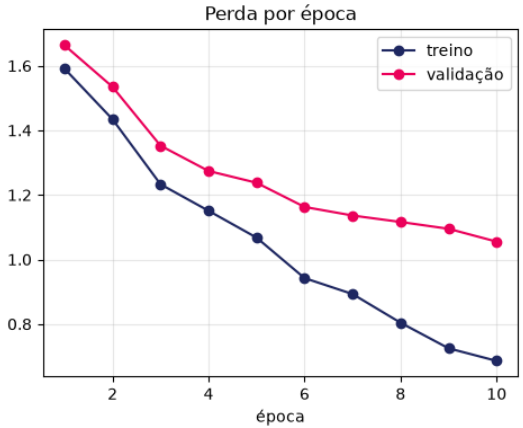
- **O modelo é uma etapa em seis.** A maior parte do tempo e do custo está nos dados e na operação.
- **O mundo muda:** nova câmera, nova iluminação, novo produto. Sem monitoramento, o modelo degrada em silêncio.
- **Hoje praticamos as etapas 3 a 5** nos notebooks.

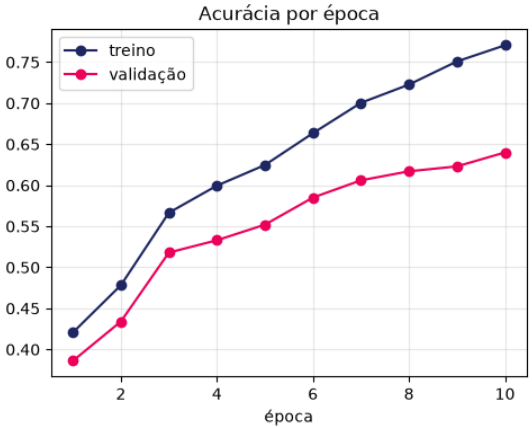
Acurácia não basta: a matriz de confusão mostra onde dói



- **Linha = classe real, coluna = palpite.** A diagonal são os acertos.
- **Nesta execução:** gato foi chamado de cachorro 41 vezes; caminhão foi chamado de carro 52 vezes.
- **Nem todo erro custa o mesmo.** Confundir gato com cachorro num app de fotos é inofensivo; confundir peça boa com defeituosa numa linha de produção, não.
- **Em produção,** este mapa define onde entra revisão humana e onde o modelo decide sozinho.

Ler a curva de treino: o que a rede está fazendo





- **Perda caindo nos dois:** a rede está aprendendo.
- **Distância entre treino e validação crescendo:** começou a decorar. Hora de parar ou regularizar.
- **Resultado real desta configuração:** 9.000 imagens, 10 épocas, CPU: 61,6% de acurácia no teste.
- **Chute aleatório:** 10%.

Com o dataset completo (50.000 imagens), mais épocas e GPU, redes desta família passam de 80%. Em aula usamos um subconjunto para o treino caber em CPU. A mecânica é a mesma; o que muda é escala.

APLICAÇÃO 1 | 50 MIN

Uma CNN classificando 10 tipos de objeto

Notebook 02_aplicacao1_cnn_cifar10. Dados → preparo → arquitetura → treino → avaliação → o que a CNN vê → registro no MLflow.

- Checkpoints a cada bloco: só avançamos quando todos chegaram
- Resultado esperado: acurácia na casa dos 60% no teste, em 2 a 6 minutos de treino em CPU
- Cada um terá um número um pouco diferente. Vamos discutir por quê

Classificar, localizar, segmentar: três perguntas diferentes

Classificação

O que há na imagem?

SAÍDA

Um rótulo por imagem.

EXEMPLOS

Tipo de documento, peça OK ou defeito, categoria do produto.

Detecção

O que há, e onde?

SAÍDA

Caixas com rótulo e confiança.

EXEMPLOS

Contar pessoas, localizar produtos na gôndola, placas de veículos.

Segmentação

Quais pixels pertencem a quê?

SAÍDA

Uma máscara pixel a pixel.

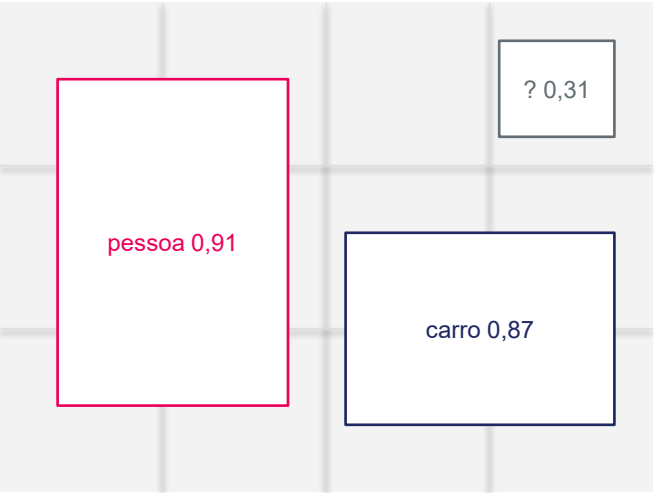
EXEMPLOS

Área de uma lesão, medir uma trinca, remover fundo.

Hoje: classificação na aplicação 1, detecção na aplicação 2.

Detecção em uma passada: a família YOLO

- **Antes (R-CNN, 2014):** propor centenas de regiões e classificar cada uma. Preciso, mas lento.
- **YOLO, "You Only Look Once" (2016):** a imagem passa uma vez pela rede, que prevê caixas, classes e confiança ao mesmo tempo. Detecção vira um problema de regressão.
- **Resultado:** velocidade de vídeo em tempo real.
- **Pré-treinado no COCO:** 80 classes do cotidiano (pessoa, carro, cadeira, xícara, cachorro...). Usamos sem treinar nada.
- **A mesma CNN por baixo.** O que muda é a cabeça: em vez de 10 probabilidades, coordenadas de caixas.



A rede divide a imagem numa grade e, para cada célula, prevê caixas com classe e confiança. Caixas fracas são descartadas pelo limiar.

Confiança e limiar: a decisão que é sua, não do modelo

- **Cada caixa vem com uma confiança de 0 a 1.** O limiar decide a partir de quanto a caixa "existe".
- **Limiar baixo:** encontra quase tudo, inclusive o que não está lá (mais recall, menos precisão).
- **Limiar alto:** só o que tem certeza; perde objetos difíceis (mais precisão, menos recall).
- **Contagem de fluxo em loja** tolera erro: limiar médio. Segurança em área restrita não tolera objeto perdido: limiar baixo e revisão humana.



limiar 0,1

acha tudo, erra mais

limiar 0,9

só o certo, perde objetos

Na aplicação 2 você vai rodar a mesma foto com dois limiares e contar quantos objetos aparecem e desaparecem.

APLICAÇÃO 2 | 50 MIN

Detectar e contar objetos em fotos e vídeo

Notebook 03_aplicacao2_deteccao_objetos. Um detector pré-treinado, fotos de exemplo, a sua própria foto, dois limiares, contagem por classe e um vídeo curto.

- Sem treinar nada: primeiro contato com reaproveitar um modelo pronto, tema da aula 4
- Medimos o tempo por imagem em CPU: por que câmeras em tempo real precisam de GPU ou de hardware dedicado

MOMENTO UAU | 15 MIN

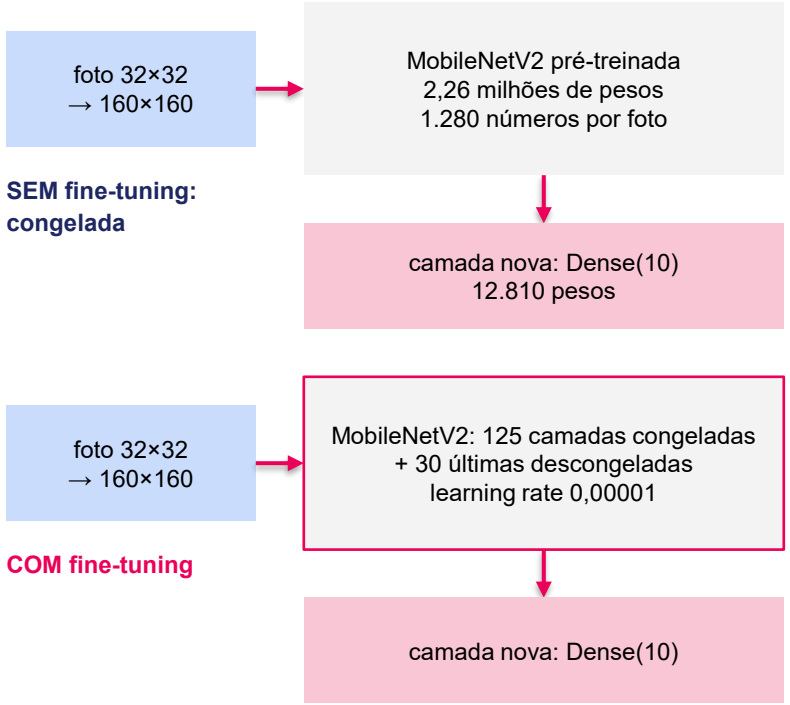
Transfer learning: reaproveitar uma rede que já enxerga

Notebook 04_uau_transfer_learning_cifar10. A mesma CIFAR-10, só 100 imagens por classe, e uma MobileNetV2 treinada na ImageNet usada de dois jeitos.

- Sem fine-tuning: a rede fica congelada; treinamos só uma camada nova no final
- Com fine-tuning: descongelamos as últimas 30 camadas, com learning rate bem pequeno
- Comparar com a CNN do zero da aplicação 1, nas mesmas 2.000 imagens de teste

Transfer learning: ensinar só o final da rede

- **ImageNet:** 1,2 milhão de fotos em 1.000 categorias. Quem treinou nela já pagou a conta mais cara.
- **As primeiras camadas são universais:** bordas, texturas, cantos servem para gato, peça ou raio-X.
- **Só o final é específico.** Trocamos a última camada por uma nova com as nossas 10 classes.
- **Sem fine-tuning:** a rede fica congelada; só a camada nova aprende (12.810 pesos).
- **Com fine-tuning:** descongelamos as últimas camadas com learning rate muito baixo. Passo grande destrói o que a rede sabia.



1.000 imagens batem 9.000: os números no mesmo teste

Abordagem	Imagens de treino	O que treina	Acurácia no teste	Tempo (CPU)
CNN do zero (aplicação 1)	9.000	356 mil pesos, 10 épocas	**59,5%**	~2 min
MobileNetV2 congelada (sem fine-tuning)	1.000	12.810 pesos, 5 épocas	**62,9%**	~3 min
MobileNetV2 com fine-tuning	1.000	últimas 30 camadas, +5 épocas	**65,1%**	+2,5 min

- **Nove vezes menos dados, nenhum treino da rede, resultado melhor.** Esse é o uau: o trabalho caro já estava feito.
- **O fine-tuning ajudou pouco aqui** porque as fotos da CIFAR-10 têm 32 pixels esticados para 160; a MobileNetV2 viu fotos nítidas. Com fotos reais do negócio, o ganho cresce.
- **Com 10 imagens por classe,** a rede congelada ainda acerta na casa de 54% (medido aqui com os mesmos vetores). A CNN do zero não aprende nada com 100 fotos.
- **Resultados reais do notebook 04,** executado em CPU; os seus vão variar um pouco.

Onde isso gera valor hoje

Inspeção visual na fábrica

O QUE O MODELO FAZ

Classificar peça boa ou defeituosa a partir de fotos da linha.

DADO NECESSÁRIO

Fotos da linha, rotuladas pelo controle de qualidade.

MÉTRICA DE NEGÓCIO

Defeitos que escapam; peças boas rejeitadas.

Loja e gôndola

O QUE O MODELO FAZ

Detectar e contar pessoas, produtos e vazios na prateleira.

DADO NECESSÁRIO

Câmeras já instaladas; rotulagem de caixas.

MÉTRICA DE NEGÓCIO

Ruptura de estoque; tempo de fila.

Documentos

O QUE O MODELO FAZ

Classificar o tipo de documento e localizar campos antes do OCR.

DADO NECESSÁRIO

Digitalizações; exemplos de cada tipo.

MÉTRICA DE NEGÓCIO

Tempo de processamento; retrabalho manual.

Quando não usar deep learning em visão

- **Uma regra simples resolve:** cor, tamanho, limiar de brilho. Uma placa "PARE" é vermelha; às vezes contar pixels vermelhos basta.
- **Poucas imagens e nenhum modelo pré-treinado para adaptar:** sem dado, a rede não tem de onde aprender (o momento uau de hoje mostra como poucas imagens bastam quando há um modelo para reaproveitar).
- **O custo de rotular supera o ganho:** milhares de fotos marcadas por especialistas raramente são baratas.
- **Latência e hardware no limite:** tempo real numa câmera barata exige engenharia que o piloto não mostra.

REGRA PRÁTICA

Comece pelo baseline mais simples que resolva. Só suba de complexidade quando o baseline provar que não basta.

Redes neurais são a ferramenta certa quando o padrão é complexo demais para ser descrito em regras, e há dados para aprendê-lo.

Case e discussão

Em grupos de 4 ou 5, 15 minutos. Depois, 15 minutos de plenária: cada grupo apresenta um caso em 2 minutos.

1	Sua empresa tem hoje um fluxo de imagens (fotos, scans, câmeras)? Que decisão depende dele e quem a toma?
2	Nesse fluxo, o que seria um erro caro? Onde você colocaria revisão humana e onde o modelo decidiria sozinho?
3	O problema é de classificação, detecção ou segmentação? Quantas imagens rotuladas vocês conseguiriam em 30 dias, e a que custo?

Critério de uma boa resposta: nomeia o dado, o erro caro e o baseline antes de nomear o modelo.

Conclusões

- Uma rede aprende repetindo um ciclo simples: palpite, erro, culpa (backprop), passo (gradiente). Nada mais.
- Learning rate, função de perda e divisão treino/validação/teste são decisões, e cada uma pode arruinar um projeto.
- Convolução reaproveita o mesmo filtro na imagem inteira: é por isso que redes conseguem ver com poucos pesos.
- Acurácia é um número; a matriz de confusão é um mapa. O mapa é que define onde entra gente.
- Treinamos do zero com 9.000 imagens e chegamos a 60%. Com 1.000 imagens e uma rede pré-treinada, 63% sem treinar a rede e 65% com fine-tuning. Reaproveitar modelos prontos volta na aula 4, com texto.

GANCHO PARA A AULA 2

Pixels são números. Palavras não são.

Uma foto já chega como 3.072 números. Um e-mail, um contrato ou um ticket de SAC chegam como texto. Como transformar palavras em números que uma rede entende, sem perder o sentido? É a aula 2: PLN, do clássico aos embeddings.

Leituras e materiais complementares

- **Goodfellow, Bengio e Courville.** Deep Learning. MIT Press, 2016. Gratuito em deeplearningbook.org (capítulos 6 e 9).
- **Stanford CS231n.** Convolutional Neural Networks for Visual Recognition. Notas de aula abertas em cs231n.github.io.
- **3Blue1Brown.** Série "Neural networks" no YouTube: a melhor intuição visual sobre gradiente e backpropagation.
- **Rumelhart, Hinton e Williams.** Learning representations by back-propagating errors. Nature, 1986.
- **Redmon et al.** You Only Look Once: Unified, Real-Time Object Detection. CVPR, 2016.
- **PyTorch.** Tutoriais oficiais em pytorch.org/tutorials, em especial "Training a Classifier".
- **Databricks.** Documentação da Free Edition e do serverless em docs.databricks.com.



Copyright © 2026 | Profa. Juliana Scudilio | Todos os direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibida sem consentimento formal, por escrito, do professor/autor.